KIGALI INDEPENDENT UNIVERSITY — ULK

SQL DATABASE SYSTEMS · PRACTICAL EXERCISES

Complete PostgreSQL solutions for Questions 1–10

Student name: _____ **Registration no.:** _____

Class / Group: _____ **Date:** ____ / ____ / 2026

How to use: Run each question's SQL in PostgreSQL in the order shown: CREATE, INSERT, UPDATE, SELECT and JOIN/report. Run each question separately in a fresh database, or use different table names if combining questions because some table names may be reused.

Important: The sample records are illustrative. The solutions follow the table names, columns and requirements in the supplied exercise sheet. Each SQL statement ends with a semicolon.

Question 1 — Library System

```
-- a. CREATE TABLES
CREATE TABLE authors (
    author_id INT PRIMARY KEY,
    author_name VARCHAR(100) NOT NULL,
    country VARCHAR(50)
);

CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    author_id INT REFERENCES authors(author_id),
    published_year INT,
    copies INT CHECK (copies >= 0)
);

-- b. INSERT DATA
INSERT INTO authors (author_id, author_name, country) VALUES
(1, 'Chinua Achebe', 'Nigeria'),
(2, 'Ngugi wa Thiong''o', 'Kenya');

INSERT INTO books (book_id, title, author_id, published_year, copies) VALUES
(1, 'Things Fall Apart', 1, 1958, 8),
(2, 'The River Between', 2, 1965, 4),
(3, 'A New Dawn', 1, 2020, 6);

-- c. UPDATE book 2 copies
UPDATE books
SET copies = 10
WHERE book_id = 2;

-- d. BOOKS PUBLISHED AFTER 2015
SELECT *
FROM books
WHERE published_year > 2015
ORDER BY title ASC;

-- e. BOOK TITLE WITH AUTHOR NAME
SELECT b.title, a.author_name
FROM books AS b
INNER JOIN authors AS a ON b.author_id = a.author_id;
```

Question 2 — Clinic Visits

```
-- a. CREATE TABLES
CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    age INT,
    district VARCHAR(50)
);

CREATE TABLE doctors (
    doctor_id INT PRIMARY KEY,
    doctor_name VARCHAR(100) NOT NULL,
    specialty VARCHAR(50)
);

CREATE TABLE visits (
    visit_id INT PRIMARY KEY,
    patient_id INT REFERENCES patients(patient_id),
    doctor_id INT REFERENCES doctors(doctor_id),
```

```
    visit_date DATE,
    fee INT
);

-- b. INSERT DATA (parents before visits)
INSERT INTO patients (patient_id, full_name, age, district) VALUES
(1, 'Jean Niyonzima', 45, 'Gasabo'),
(2, 'Aline Mukamana', 32, 'Kicukiro');

INSERT INTO doctors (doctor_id, doctor_name, specialty) VALUES
(1, 'Dr. Eric', 'General Medicine'),
(2, 'Dr. Grace', 'Pediatrics');

INSERT INTO visits (visit_id, patient_id, doctor_id, visit_date, fee) VALUES
(1, 1, 1, '2026-10-01', 10000),
(2, 2, 2, '2026-10-03', 12000),
(3, 1, 2, '2026-10-05', 9000);

-- c. UPDATE visit 3 fee
UPDATE visits
SET fee = 15000
WHERE visit_id = 3;

-- d. PATIENTS FROM GASABO OLDER THAN 40
SELECT *
FROM patients
WHERE district = 'Gasabo' AND age > 40;

-- e. PATIENT, DOCTOR AND VISIT DATE
SELECT p.full_name, d.doctor_name, v.visit_date
FROM visits AS v
INNER JOIN patients AS p ON v.patient_id = p.patient_id
INNER JOIN doctors AS d ON v.doctor_id = d.doctor_id;
```

Question 3 — Shop Orders

```
-- a. CREATE TABLES
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) UNIQUE
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    unit_price INT,
    stock_qty INT
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    product_id INT REFERENCES products(product_id),
    quantity INT CHECK (quantity > 0),
    order_date DATE
);

-- b. INSERT DATA
INSERT INTO customers (customer_id, full_name, phone) VALUES
(1, 'Peter Mugisha', '0788000001'),
(2, 'Sarah Uwase', '0788000002');

INSERT INTO products (product_id, product_name, unit_price, stock_qty) VALUES
(1, 'Keyboard', 15000, 30),
(2, 'Mouse', 5000, 40),
(3, 'USB Flash Drive', 3000, 25);

INSERT INTO orders (order_id, customer_id, product_id, quantity, order_date) VALUES
(1, 1, 1, 2, '2026-10-10'),
(2, 2, 2, 3, '2026-10-11'),
(3, 1, 3, 5, '2026-10-12');

-- c. REDUCE STOCK OF PRODUCT 1 BY 5
UPDATE products
SET stock_qty = stock_qty - 5
WHERE product_id = 1;

-- d. PRODUCTS PRICED BETWEEN 1000 AND 5000
SELECT *
FROM products
WHERE unit_price BETWEEN 1000 AND 5000;

-- e. TOTAL QUANTITY ORDERED FOR EACH PRODUCT
SELECT p.product_name, COALESCE(SUM(o.quantity), 0) AS total_quantity_ordered
FROM products AS p
```

```
LEFT JOIN orders AS o ON p.product_id = o.product_id
GROUP BY p.product_id, p.product_name
ORDER BY p.product_name;
```

Question 4 — Student Marks

```
-- a. CREATE TABLES
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    gender CHAR(1) CHECK (gender IN ('M', 'F'))
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    credits INT
);

CREATE TABLE results (
    result_id INT PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    mark INT CHECK (mark BETWEEN 0 AND 100)
);

-- b. INSERT DATA
INSERT INTO students (student_id, full_name, gender) VALUES
(1, 'David Nkurunziza', 'M'),
(2, 'Alice Uwimana', 'F'),
(3, 'Samuel Habimana', 'M');

INSERT INTO courses (course_id, course_name, credits) VALUES
(1, 'Database Systems', 4),
(2, 'Programming', 5);

INSERT INTO results (result_id, student_id, course_id, mark) VALUES
(1, 1, 1, 65),
(2, 2, 1, 72),
(3, 3, 2, 48),
(4, 1, 2, 80);

-- c. ADD 5 MARKS TO EVERY RESULT IN COURSE 1
-- LEAST(100, ...) keeps marks within the CHECK constraint.
UPDATE results
SET mark = LEAST(mark + 5, 100)
WHERE course_id = 1;

-- d. RESULTS WITH MARK 50 OR MORE, HIGHEST FIRST
SELECT *
FROM results
WHERE mark >= 50
ORDER BY mark DESC;

-- e. AVERAGE MARK FOR EACH COURSE
SELECT c.course_name, ROUND(AVG(r.mark), 2) AS average_mark
FROM courses AS c
INNER JOIN results AS r ON c.course_id = r.course_id
GROUP BY c.course_id, c.course_name;
```

Question 5 — Bank Accounts

```
-- a. CREATE TABLES
CREATE TABLE clients (
    client_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    national_id VARCHAR(16) UNIQUE
);

CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    client_id INT REFERENCES clients(client_id),
    account_type VARCHAR(20) CHECK (account_type IN ('Savings', 'Current')),
    balance INT CHECK (balance >= 0)
);

-- b. INSERT DATA
INSERT INTO clients (client_id, full_name, national_id) VALUES
(1, 'Michael Rukundo', '1199080000000001'),
(2, 'Diane Uwera', '1199080000000002');

INSERT INTO accounts (account_id, client_id, account_type, balance) VALUES
(1, 1, 'Savings', 80000),
(2, 1, 'Current', 5000),
(3, 2, 'Savings', 25000);
```

```sql
-- c. DEPOSIT 50000 INTO ACCOUNT 1
UPDATE accounts
SET balance = balance + 50000
WHERE account_id = 1;

-- d. ACCOUNTS WITH BALANCE BELOW 10000
SELECT *
FROM accounts
WHERE balance < 10000;

-- e. EACH CLIENT WITH TOTAL BALANCE OF THEIR ACCOUNTS
SELECT c.full_name, COALESCE(SUM(a.balance), 0) AS total_balance
FROM clients AS c
LEFT JOIN accounts AS a ON c.client_id = a.client_id
GROUP BY c.client_id, c.full_name;
```

Question 6 — Hotel Bookings

```sql
-- a. CREATE TABLES
CREATE TABLE guests (
    guest_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    nationality VARCHAR(50)
);

CREATE TABLE rooms (
    room_no INT PRIMARY KEY,
    room_type VARCHAR(20) CHECK (room_type IN ('Single', 'Double', 'Deluxe')),
    price_per_night INT
);

CREATE TABLE bookings (
    booking_id INT PRIMARY KEY,
    guest_id INT REFERENCES guests(guest_id),
    room_no INT REFERENCES rooms(room_no),
    check_in DATE,
    nights INT CHECK (nights > 0)
);

-- b. INSERT DATA
INSERT INTO guests (guest_id, full_name, nationality) VALUES
(1, 'Alex Mutesi', 'Rwandan'),
(2, 'John Smith', 'Kenyan');

INSERT INTO rooms (room_no, room_type, price_per_night) VALUES
(101, 'Single', 20000),
(102, 'Double', 35000),
(103, 'Deluxe', 60000);

INSERT INTO bookings (booking_id, guest_id, room_no, check_in, nights) VALUES
(1, 1, 101, '2026-11-10', 2),
(2, 2, 103, '2026-11-12', 3);

-- c. INCREASE PRICE OF ALL DELUXE ROOMS BY 10%
UPDATE rooms
SET price_per_night = ROUND(price_per_night * 1.10)
WHERE room_type = 'Deluxe';

-- d. ROOMS THAT HAVE NEVER BEEN BOOKED
-- LEFT JOIN is required to retain rooms without a matching booking.
SELECT r.*
FROM rooms AS r
LEFT JOIN bookings AS b ON r.room_no = b.room_no
WHERE b.booking_id IS NULL;

-- e. GUEST, ROOM NUMBER AND TOTAL COST
SELECT g.full_name, r.room_no,
       (b.nights * r.price_per_night) AS total_cost
FROM bookings AS b
INNER JOIN guests AS g ON b.guest_id = g.guest_id
INNER JOIN rooms AS r ON b.room_no = r.room_no;
```

Question 7 — Bus Transport

```sql
-- a. CREATE TABLES
CREATE TABLE buses (
    bus_id INT PRIMARY KEY,
    plate_no VARCHAR(10) UNIQUE NOT NULL,
    capacity INT
);

CREATE TABLE routes (
    route_id INT PRIMARY KEY,
    origin VARCHAR(50),
    destination VARCHAR(50),
```

```
    fare INT
);

CREATE TABLE trips (
    trip_id INT PRIMARY KEY,
    bus_id INT REFERENCES buses(bus_id),
    route_id INT REFERENCES routes(route_id),
    trip_date DATE,
    passengers INT
);

-- b. INSERT DATA
INSERT INTO buses (bus_id, plate_no, capacity) VALUES
(1, 'RAD123A', 50),
(2, 'RAE456B', 45);

INSERT INTO routes (route_id, origin, destination, fare) VALUES
(1, 'Kigali', 'Huye', 3000),
(2, 'Kigali', 'Musanze', 2500);

INSERT INTO trips (trip_id, bus_id, route_id, trip_date, passengers) VALUES
(1, 1, 1, '2026-10-15', 40),
(2, 2, 2, '2026-10-16', 35),
(3, 1, 1, '2026-10-17', 45);

-- c. CHANGE KIGALI TO HUYE FARE TO 3500
UPDATE routes
SET fare = 3500
WHERE origin = 'Kigali' AND destination = 'Huye';

-- d. ROUTES STARTING IN KIGALI, LOWEST FARE FIRST
SELECT *
FROM routes
WHERE origin = 'Kigali'
ORDER BY fare ASC;

-- e. NUMBER OF TRIPS AND TOTAL PASSENGERS FOR EACH ROUTE
SELECT r.route_id, r.origin, r.destination,
       COUNT(t.trip_id) AS number_of_trips,
       COALESCE(SUM(t.passengers), 0) AS total_passengers
FROM routes AS r
LEFT JOIN trips AS t ON r.route_id = t.route_id
GROUP BY r.route_id, r.origin, r.destination;
```

Question 8 — Restaurant Sales

```
-- a. CREATE TABLES
CREATE TABLE waiters (
    waiter_id INT PRIMARY KEY,
    waiter_name VARCHAR(100) NOT NULL
);

CREATE TABLE menu_items (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100) NOT NULL,
    category VARCHAR(30) CHECK (category IN ('Food', 'Drinks')),
    price INT
);

CREATE TABLE sales (
    sale_id INT PRIMARY KEY,
    waiter_id INT REFERENCES waiters(waiter_id),
    item_id INT REFERENCES menu_items(item_id),
    qty INT,
    sale_date DATE
);

-- b. INSERT DATA
INSERT INTO waiters (waiter_id, waiter_name) VALUES
(1, 'Eric'),
(2, 'Grace');

INSERT INTO menu_items (item_id, item_name, category, price) VALUES
(1, 'Chicken', 'Food', 5000),
(2, 'Water', 'Drinks', 1000),
(3, 'Fresh Juice', 'Drinks', 2000),
(4, 'Rice', 'Food', 2500);

INSERT INTO sales (sale_id, waiter_id, item_id, qty, sale_date) VALUES
(1, 1, 1, 2, '2026-10-20'),
(2, 2, 2, 5, '2026-10-20'),
(3, 1, 3, 3, '2026-10-21'),
(4, 2, 4, 2, '2026-10-21');

-- c. CHANGE PRICE OF ITEM 3 TO 2500
UPDATE menu_items
```

```
SET price = 2500
WHERE item_id = 3;

-- d. DISPLAY DRINKS, MOST EXPENSIVE FIRST
SELECT *
FROM menu_items
WHERE category = 'Drinks'
ORDER BY price DESC;

-- e. EACH WAITER WITH TOTAL MONEY SOLD
SELECT w.waiter_name,
       COALESCE(SUM(s.qty * m.price), 0) AS total_money_sold
FROM waiters AS w
LEFT JOIN sales AS s ON w.waiter_id = s.waiter_id
LEFT JOIN menu_items AS m ON s.item_id = m.item_id
GROUP BY w.waiter_id, w.waiter_name;
```

Question 9 — Pharmacy Stock

```
-- a. CREATE TABLES
CREATE TABLE suppliers (
    supplier_id INT PRIMARY KEY,
    supplier_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15)
);

CREATE TABLE medicines (
    medicine_id INT PRIMARY KEY,
    medicine_name VARCHAR(100) NOT NULL,
    expiry_date DATE,
    stock INT DEFAULT 0
);

CREATE TABLE supplies (
    supply_id INT PRIMARY KEY,
    supplier_id INT REFERENCES suppliers(supplier_id),
    medicine_id INT REFERENCES medicines(medicine_id),
    quantity INT,
    supply_date DATE
);

-- b. INSERT DATA (one medicine has an expiry date in the past)
INSERT INTO suppliers (supplier_id, supplier_name, phone) VALUES
(1, 'Kigali Medical Supplies', '0788111001'),
(2, 'HealthPlus Ltd', '0788222002');

INSERT INTO medicines (medicine_id, medicine_name, expiry_date, stock) VALUES
(1, 'Paracetamol', '2024-01-01', 50),
(2, 'Amoxicillin', '2027-06-30', 80),
(3, 'Vitamin C', '2027-12-31', 40);

INSERT INTO supplies (supply_id, supplier_id, medicine_id, quantity, supply_date) VALUES
(1, 1, 1, 50, '2026-10-01'),
(2, 2, 2, 80, '2026-10-02'),
(3, 1, 3, 40, '2026-10-03');

-- c. ADD 100 UNITS TO MEDICINE 2
UPDATE medicines
SET stock = stock + 100
WHERE medicine_id = 2;

-- d. MEDICINES THAT HAVE ALREADY EXPIRED
SELECT *
FROM medicines
WHERE expiry_date < CURRENT_DATE;

-- e. SUPPLIER, MEDICINE AND QUANTITY SUPPLIED
SELECT s.supplier_name, m.medicine_name, sp.quantity
FROM supplies AS sp
INNER JOIN suppliers AS s ON sp.supplier_id = s.supplier_id
INNER JOIN medicines AS m ON sp.medicine_id = m.medicine_id;
```

Question 10 — Company Employees

```
-- a. CREATE TABLES
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) UNIQUE
);

CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    salary INT,
    dept_id INT REFERENCES departments(dept_id),
```

```
    manager_id INT REFERENCES employees(emp_id)
);

-- b. INSERT DATA
INSERT INTO departments (dept_id, dept_name) VALUES
(1, 'IT'),
(2, 'Finance');

-- Insert the manager first. A NULL manager_id means this employee
-- does not report to another employee.
INSERT INTO employees (emp_id, full_name, salary, dept_id, manager_id) VALUES
(1, 'David Manager', 900000, 1, NULL);

INSERT INTO employees (emp_id, full_name, salary, dept_id, manager_id) VALUES
(2, 'Alice Staff', 550000, 1, 1),
(3, 'Peter Staff', 480000, 1, 1),
(4, 'Grace Accountant', 600000, 2, 1);

-- c. GIVE EVERY EMPLOYEE IN IT A 5% SALARY INCREASE
UPDATE employees
SET salary = ROUND(salary * 1.05)
WHERE dept_id = (
    SELECT dept_id
    FROM departments
    WHERE dept_name = 'IT'
);

-- d. EMPLOYEES EARNING MORE THAN 500000, HIGHEST FIRST
SELECT *
FROM employees
WHERE salary > 500000
ORDER BY salary DESC;

-- e. EMPLOYEE NAME NEXT TO MANAGER NAME (SELF JOIN)
-- LEFT JOIN keeps the top-level manager, whose manager_id is NULL.
SELECT e.full_name AS employee_name,
       m.full_name AS manager_name
FROM employees AS e
LEFT JOIN employees AS m ON e.manager_id = m.emp_id
ORDER BY e.emp_id;
```